Catmull-Clark algoritmus implementálása

Share

butterfly_subdivision.pde PDE

drawModel.pde PDE

Interaction.pde PDE

loop_subdivision.pde PDE

MeshLoader.pde PDE

all_of_them_subdivision_main.pde PDE

add meg a catmull-clark algoritmust és módosítsd a kódot, hogy működjön az egész

Catmull-Clark algoritmus és a feltöltött kód javítása folyamatban.

Először átnézem a feltöltött fájlokat.

Ran 3 commands, added to memory, shared files

Elkészült a Catmull-Clark algoritmus, és átalakítottam a kódot úgy, hogy az egész sketch együtt fusson. Processing nem volt a környezetemben, ezért lefuttatni nem tudtam, csak átnéztem a kódot. A kocka sarokpontjára kézzel kiszámoltam a Catmull-Clark képletet, és a jól ismert 5/9-et kaptam.

Új fájlok

- `catmull_clark_subdivision.pde`: az algoritmus maga, tetszőleges soklapra (n-szögre), nyitott éleket is kezelve.
  - A lappontok a lap csúcsainak átlagai.
  - A belső élpontok `(A+B+F1+F2)/4`-ként jönnek ki, a nyitott éleké az él felezőpontja.
  - A belső csúcsok `(F + 2R + (n-3)P) / n` szerint mozdulnak, a szélső csúcsok `3/4 P + 1/8` a két szélső szomszédra.
  - Minden lapból n négyszög lesz.
- `mesh_utils.pde`: közös élnyilvántartás (`EdgeData`) és `ensureTriangles()`.
- `loop_subdivision.pde`: ez üres volt, de a main `applyLoopSubdivision()`-t hívta, ezért megírtam a szabványos Loop-algoritmust.

Módosított fájlok

- `MeshLoader.pde`: a betöltő eddig háromszögekre bontotta a négyszögeket, így a Catmull-Clark sosem kapott volna négyszöget. Mostantól a lapok eredeti alakjukban

maradnak. Kezeli a negatív OBJ-indexeket, és ha nincs `cube.obj`, beépített kockát használ (ez nem volt feltöltve).

- `drawModel.pde`: két hibát javítottam.
  - Dupla `beginShape` volt benne, és csak az első 3 csúcsot rajzolta ki lapokként.
  - Most háromszögeket, négyszögeket és n-szögeket is kirajzol, és normálisokat számol (lapnormális, illetve a 3-as módban csúcsnormális).
- `all_of_them_subdivision_main.pde`: kikerült az `enum Shape`, mert a lap hossza alapján rajzolunk. A szintek építése külön `buildLevels()` függvénybe került.
- `butterfly_subdivision.pde`: két kis módosítás: a `shape = ...` sor törölve, és a függvény elején `ensureTriangles()` fut.
- `Interaction.pde`: új **4**-es gomb, amellyel futás közben váltasz Catmull-Clark, Loop és Butterfly között. A kijelzőn látszik az algoritmus neve és a lapok száma.

Az induló algoritmus a Catmull-Clark. A sketch mappájának neve legyen `all_of_them_subdivision_main`, a `cube.obj` pedig a `data/` almappába kerüljön. Ha egy modell fordítva (sötéten) világít, akkor az OBJ lapjainak csúcssorrendje ellentétes a vártnál (a program óramutató-járással ellentétes sorrendet feltételez).

Download all

hol tudom kiválasztani, hogy melyik algoritmust használjam?

Két helyen:

1. **Futás közben:** nyomd meg a **4-es gombot.** Ez körbeléptet a három algoritmuson (Catmull-Clark → Loop → Butterfly), és minden váltásnál újraszámolja a szinteket. A kiválasztott algoritmus neve a bal felső sarokban látszik ("Algoritmus: ...").
2. **Kódban, az induló algoritmus:** az `all_of_them_subdivision_main.pde` elején ez a sor:

```java
int algorithm = 0;
```

Itt a `0` a Catmull-Clark, az `1` a Loop, a `2` a Butterfly. Ha például Loop-pal akarsz indulni, írj `1`-et.

hogyan lehetne megoldani, hogy a forgatás ne legyen ilyen darabos?

Building the cached shape for each render mode ▸

A darabosságot valószínűleg az okozza, hogy a `drawModel()` minden képkockánál újra átadja a GPU-nak az egész hálót (több ezer `vertex()` hívás, az élrajzolással együtt). A megoldás az, hogy a hálót egyszer felépítem egy `PShape`-be, és rajzoláskor csak ezt a kész objektumot rajzolom ki. Átírom a fájlokat.

Ran a command, shared files ▸

Átírtam a rajzolást, így a forgatásnak sokkal simábbnak kell lennie. Futtatni itt nem tudtam, ezért próbáld ki nálad.

Mi változott

- A `drawModel.pde`-ben új `buildModelShape()` függvény egyszer felépíti a hálót egy `PShape`-be. A stílus (kitöltés, vonal) és a normálisok is a `PShape`-ben vannak.
- A `drawModel()` már csak a kész objektumot rajzolja ki, ezért forgatáskor nincs több ezer `vertex()` hívás képkockánként.
- Az `Interaction.pde`-ben a `buildModelShape()` lefut szintváltáskor (`switchLevel`) és a 3-as gombbal történő módváltáskor, tehát a váltások utáni első képkocka kicsit lassabb lehet, de utána gyors.

Ha még mindig darabos

- Az 1-es mód (háló + kitöltés) és a 0-s mód (áttetsző) vonalakat is rajzol, ami a legdrágább. A 2-es mód (`noStroke`) a leggyorsabb.
- A 4. szintű háló Catmull-Clarkkal 1536 lap, Loop-pal 3072 háromszög. Ha nagyobb modellt töltesz be, érdemes a `MAX_LEVELS`-t 3-ra venni a main fájlban.
- Ha az egér mozgatása után is "ugrál" a forgás, lehet simítást (inerciát) adni neki: a `rotX`/`rotY` közvetlen állítása helyett célértékeket állítasz, és a `draw()`-ban `lerp`-pel közelíted őket. Ezt megírom, ha kéred.

Download all